Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

Звіт

Про виконання лабораторної роботи №5

«УМОВНІ ОПЕРАТОРИ»

Виконав:
Студент групи ФЕП-13
Шмирко В.М.

Перевірив:
ас. Васюта В. М.

Львів 2026

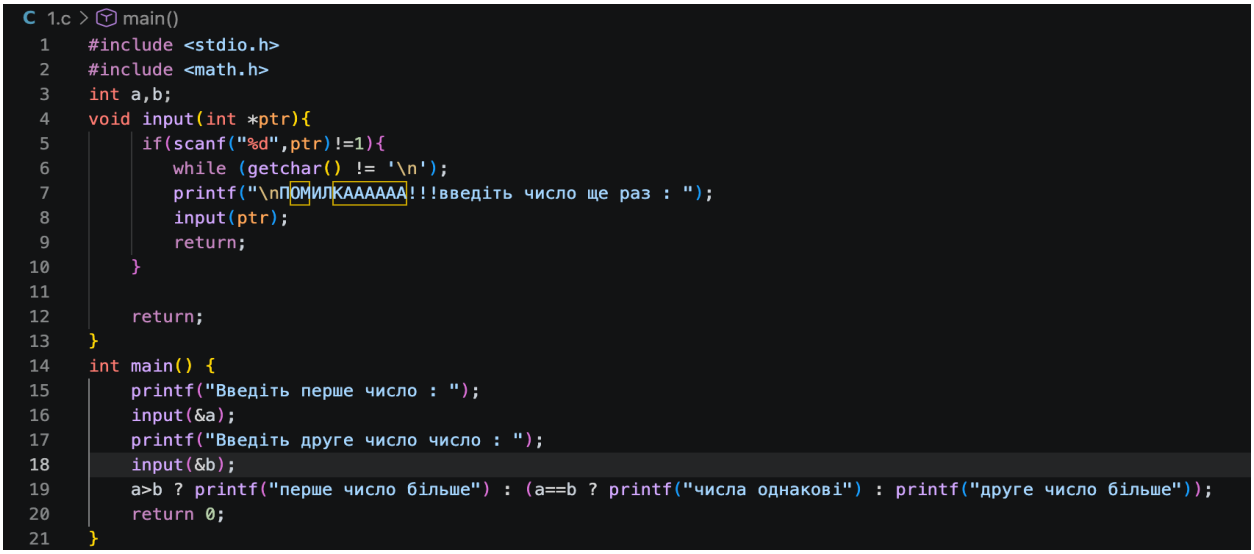
Мета роботи:

Вивчити поняття і застосування умовних операторів.

Хід роботи

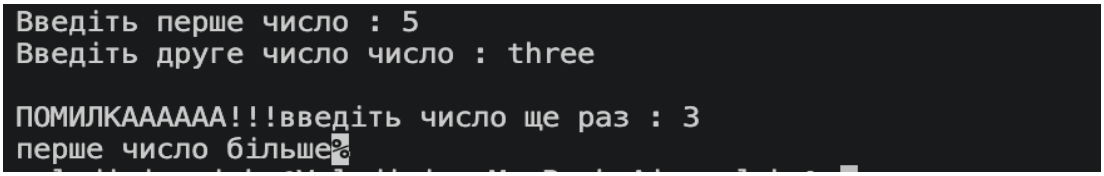
1. Програма порівняння двох чисел за допомогою тернарного оператора «?» з обробкою помилок введення.

Реалізовано функцію input(), яка рекурсивно запитує введення, доки користувач не введе коректне ціле число. Порівняння виконується одним рядком через вкладені тернарні оператори. Це дозволяє уникнути громіздких if-else конструкцій і робить код компактнішим.



```c
#include <stdio.h>
#include <math.h>
int a,b;
void input(int *ptr){
     if(scanf("%d",ptr)!=1){
        while (getchar() != '\n');
        printf("\nПОМИЛКАААААА!!!введіть число ще раз : ");
        input(ptr);
        return;
     }

     return;
}
int main() {
    printf("Введіть перше число : ");
    input(&a);
    printf("Введіть друге число число : ");
    input(&b);
    a>b ? printf("перше число більше") : (a==b ? printf("числа однакові") : printf("друге число більше"));
    return 0;
}
```



```
Введіть перше число : 5
Введіть друге число число : three

ПОМИЛКАААААА!!!введіть число ще раз : 3
перше число більше%
```

При введенні 5 і «three» програма повідомляє про помилку і просить ввести число ще раз; після коректного введення 3 виводить «перше число більше».

2. Програма визначення типу трикутника за довжинами сторін (рівносторонній / рівнобедрений / різносторонній та прямокутний / гострокутний / тупокутний).

Спочатку перевіряється нерівність трикутника. Далі за допомогою if-else-if визначається вид за сторонами, а після цього — тип кутів через порівняння квадратів сторін. Для зручності введення реалізовано окремі функції для int і float з обробкою помилок.

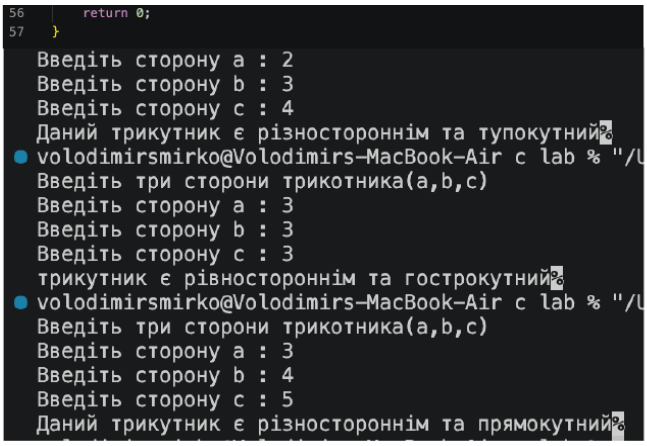
```c
#include <stdio.h>
void i_input(int *ptr){
     if(scanf("%d",ptr)!=1){
        while (getchar() != '\n');
        printf("\nПОМИЛКАААААА!!!введіть число ще раз : ");
        i_input(ptr);
        return;
     }

     return;
}void f_input(float *ptr){
     if(scanf("%f",ptr)!=1){
        while (getchar() != '\n');
        printf("\nПОМИЛКАААААА!!!введіть число ще раз : ");
        f_input(ptr);
        return;
     }

     return;
}
float a,b,c;
int main() {
    printf("Введіть три сторони трикотника(a,b,c)\n");
    printf("Введіть сторону a : ");f_input(&a);
    printf("Введіть сторону b : ");f_input(&b);
    printf("Введіть сторону c : ");f_input(&c);

    if((a+b>c)&&(a+c>b)&&(c+b>a)){
        if(a==b&&b==c){
            printf("трикутник є рівностороннім та ");
        }
        else if(a==b||c==b){
            printf("трикутник є рівнобедреним та ");
        }
        else{
        printf("Даний трикутник є різностороннім та ");
        }
        float max=a,sum=0;
        if(b>max)max=b;
        if(c>max)max=c;
        sum=a*a+b*b+c*c-max*max;

        if(sum==max*max){
            printf("прямокутний");
        }
        else if(sum>max*max){
            printf("гострокутний");
        }
        else{
            printf("тупокутний");
        }
    }
    else {
        printf("Такого трикутника не існує");
    }
    return 0;
}
```

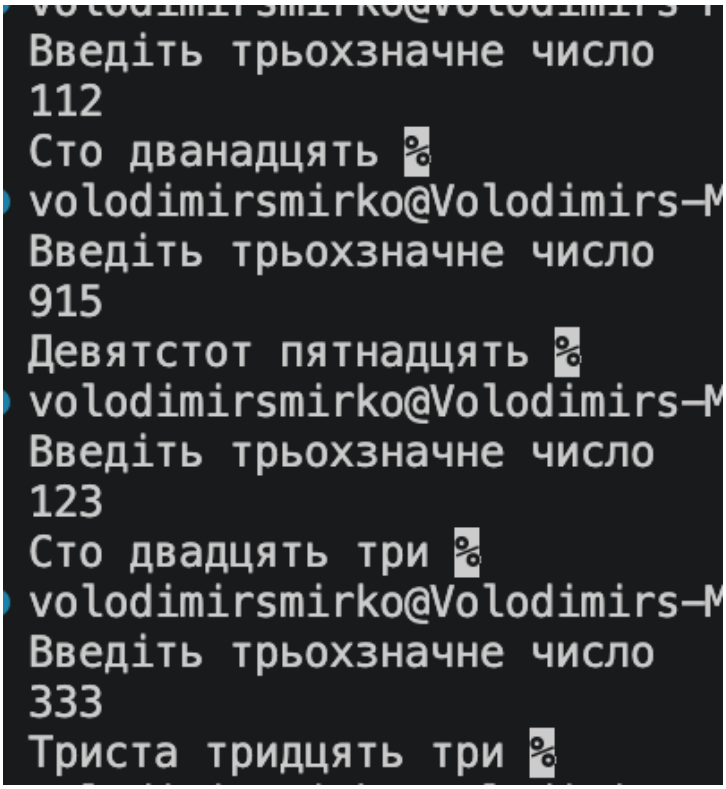


Результати тестів: 2 3 4 -> різносторонній тупокутний; 3 3 3 -> рівносторонній гострокутний; 3 4 5 -> різносторонній прямокутний.

3. Програма, що перетворює тризначне число у текстовий запис українською мовою.

Використано вкладені switch-конструкції: окремий switch для сотень, окремий для десятків (з особливим випадком для чисел 11–19) і switch для одиниць. Такий підхід виявився значно зручнішим і читабельнішим, ніж довгий ланцюжок if-else-if. Код і демонстрація роботи доступні за посиланням:

https://onlinegdb.com/yv6EFI-zF



Приклади: 112 -> «Сто дванадцять»; 915 -> «Девятсот пятнадцять»; 123 -> «Сто двадцять три»; 333 -> «Триста тридцять три».

Алгоритм завдання 1 (тернарний оператор)

1. Оголосити дві цілі змінні a і b та функцію input() з рекурсивною обробкою помилок введення.
2. Зчитати перше і друге число через input().
3. Виконати порівняння одним рядком: a > b ? «перше більше» : (a == b ? «однакові» : «друге більше»).
4. Завершити програму.

Алгоритм завдання 2 (тип трикутника)

1. Зчитати три дійсні сторони a, b, c з перевіркою коректності введення.
2. Перевірити нерівність трикутника (a+b>c && a+c>b && b+c>a). Якщо ні — вивести повідомлення про неіснування.
3. Визначити вид за сторонами: усі рівні -> рівносторонній; дві рівні -> рівнобедрений; інакше -> різносторонній.
4. Знайти найбільшу сторону і порівняти max² з сумою квадратів двох інших: рівність -> прямокутний; менше -> гострокутний; більше ->

тупокутний.
5. Завершити програму.

Алгоритм завдання 3 (число прописом)

1. Зчитати тризначне число з перевіркою діапазону 100–999.
2. switch(a/100) — вивести назву сотень.
3. Якщо десятки == 1 — окремий switch для чисел 11–19 і завершити.
4. Інакше: switch для десятків (20–90) + switch для одиниць (1–9).
5. Завершити програму.

Висновок

Умовні оператори в мові C дозволяють гнучко керувати потоком виконання програми залежно від даних. Тернарний оператор «?» виявився зручним для коротких порівнянь — один рядок замість кількох if-else, що особливо помітно при порівнянні двох чисел. Для визначення типу трикутника природніше виглядала конструкція if-else-if, бо умови тут складніші і потребують кількох перевірок поспіль.

Найбільш цікавим і практичним виявився оператор switch (і вкладені switch). Коли потрібно обробляти багато дискретних варіантів (сотні, десятки, одиниці числа), switch значно зручніший і читабельніший за довгий ланцюг if-else-if: код стає структурованішим, менше ризику помилок з фігурними дужками, і легше додавати нові case. Саме завдяки switch програма «число прописом» вийшла відносно компактною і зрозумілою, хоча й вимагала окремого випадку для чисел 11–19.